Research Design and Pilot Study Report

Alla Krylova 196722
Oleksandr Nychyporchuk 196659
Kiryl Pahkevich 196687
Pavel Khmialeuski 197055

1 Research project

1.1 Title

The impact of service-mesh solutions on the efficiency of microservice applications

1.2 Supervisor

Krzysztof Gierłowski (KT)

1.3 Goals and short description

Currently, a significant portion of applications deployed in cloud environments utilize microservices architecture. This type of architecture requires decomposing application functionality into component elements, implemented by individual microservices, and then deploying them in an environment that allows the microservices to communicate efficiently and reliably. This task is most often accomplished by orchestration platforms such as Kubernetes. Maintaining the separation and security of applications sharing the same deployment platform is also a key aspect of this type of deployment. Service mesh solutions are often used for this purpose, as they can automatically create secure communication environments for specific applications.

The aim of this project is to analyze and experimentally test the impact of using service mesh solutions on the efficiency (and particularly performance) of an application composed of multiple microservices.

2 Research design

2.1 Research goal

The goal of this research is to empirically evaluate and compare the performance overhead introduced by different service mesh implementations (specifically Istio and Linkerd) in a custom microservices application running on Kubernetes, compared to a baseline deployment without any service mesh. The custom application includes heterogeneous services: AI agent service, media processing (video, digital filters), and mathematical computation modules (differential equations, integration, permutations, Fibonacci). The research aims to quantify the trade-offs between security features (mTLS) and performance metrics (latency, throughput, CPU/memory consumption, CPU clock scaling) across these implementations.

2.2 Research gap

The systematic literature review identified 12 core articles on service mesh performance. Key findings from the SLR indicate:

1. Service meshes introduce measurable latency and resource overhead, primarily due to sidecar proxies and mTLS [1], [2].
2. Linkerd generally exhibits lower overhead than Istio under high load [3], but Istio remains the standard for complex traffic management.
3. Newer eBPF-based architectures show promise in reducing overhead [1], [4].

4. Most studies focus on either single service mesh implementations or compare different meshes using standard benchmarks. However, there is a lack of controlled, reproducible experiments that compare multiple service meshes against a common baseline using a custom, heterogeneous microservice application that stresses different resource types (CPU-intensive math, I/O-oriented media). Furthermore, few studies systematically vary security feature activation (mTLS on/off) to quantify the security-performance trade-off across meshes.

Our research will fill this gap by conducting a controlled experiment that measures the performance impact of adding different service meshes (Istio, Linkerd) to a purpose-built microservice application (designed to isolate throughput, CPU clock scaling, RAM usage, and latency), with and without security features enabled, and compares them against a no-mesh baseline.

2.3 Research questions

RQ1) How do different service mesh implementations (Istio vs. Linkerd) compare in terms of average latency and throughput relative to a baseline deployment without any service mesh, when running a heterogeneous microservice application?

RQ2) What is the CPU and memory consumption overhead of Istio and Linkerd (including sidecar proxies and control planes) under increasing request loads, and how do they compare across different service types (math, media, AI)?

RQ3) What is the additional performance overhead specifically attributable to mTLS when enabled versus disabled within each service mesh, and how does this overhead differ between Istio and Linkerd?

2.4 Research hypotheses

RQ	Hypothesis (H)	Null Hypothesis (H_0)
RQ1	H_1: Both Istio and Linkerd increase latency and reduce throughput compared to baseline, but Linkerd has lower latency and higher throughput than Istio under high load across all service types.	H_{01}: There is no statistically significant difference in latency or throughput between any pair of configurations (baseline, Istio, Linkerd).
RQ2	H_2: Linkerd consumes fewer CPU and memory resources than Istio, and both consume significantly more than the baseline (≥20% additional) - with the largest overhead observed on mathematical services (CPU-bound) and media services (memory-bound).	H_{02}: CPU and memory usage do not differ significantly between Istio, Linkerd, and baseline for any service type.
RQ3	H_3: Enabling mTLS adds measurable overhead in both meshes, but the relative overhead (percentage increase) is similar for Istio and Linkerd.	H_{03}: mTLS does not introduce significant additional performance overhead beyond the base sidecar in either mesh, or the overhead differs substantially between meshes.

All hypotheses are falsifiable and will be tested with statistical significance ($\alpha = 0.05$).

2.5 Research subjects and sample

In the context of this performance experiment, the subjects are the experimental configurations of the custom microservice application under test. Each configuration is a combination of:

- Service mesh presence: None (baseline) vs. Istio vs. Linkerd
- mTLS setting (if mesh): Disabled vs. Enabled
- Load level: Low (25 req/s), Medium (50 req/s), High (100 req/s)
- Service type: AI Service, Media (Video + Filters), Mathematical (Differential Equations, Integration, Permutations, Fibonacci)

Sample: We will use a full factorial design: 1 (baseline) + 2 (meshes: Istio, Linkerd) × 2 (mTLS on/off) × 3 load levels = 13 configurations. Each configuration will be tested 10 times across 3 service types ⟶ total experimental runs = 13 × 10 × 3 = 390. This accounts for service-type heterogeneity.

Qualification criteria for inclusion: The custom application must be deployed as a set of containerized microservices, each exposing a REST endpoint. The infrastructure must be a dedicated Kubernetes cluster. The application is designed to stress different system resources:
- **AI Service**: heavy compute, requires load caps, strict isolation.
- **Media Services**: video compression, frame splitting, matrix filters (memory and I/O).
- **Mathematical Modules**: CPU-intensive (differential equations, integration, permutations $O(n!)$, Fibonacci for clock scaling).

Sampling method: Non-probabilistic - we select the services that represent typical microservice communication patterns (request-response, chain calls, parallel fan-out).

2.6 Operationalization - variables

Variable type	Name	Operational definition / Measurement
Independent	Service mesh type	Categorical: `baseline` (no mesh), `Istio`, `Linkerd`
Independent	mTLS setting	Categorical: `disabled` vs. `enabled` (only applied to mesh conditions)
Independent	Request load	Continuous: offered load in requests per second (req/s), varied at 25, 50, 100 req/s
Dependent	Average latency	Mean request-response time (ms) per service endpoint over a 60-second steady-state window, reported as p50, p95, p99 percentiles
Dependent	Throughput	Maximum sustained requests per second achieved per service before error rate exceeds 1%
Dependent	CPU usage	Average CPU millicores consumed by mesh sidecars + application containers, separated per service
Dependent	Memory usage	Average resident memory (MiB) consumed, separated per service
Dependent	CPU clock scaling	Observed CPU frequency scaling (GHz) on math nodes under Fibonacci load
Confounding	Network conditions	Controlled by running all tests in the same isolated cluster on identical hardware; monitor for external interference
Confounding	Application version	Fixed version of the custom application across all runs
Confounding	Kubernetes version	Fixed version across all runs

Hidden	Garbage collection cycles	Minimized by running each test for a sufficient duration (warm-up + measurement) and averaging over repetitions
Hidden	Node heterogeneity	Use identical instances for all worker nodes

2.7 Research methods

This research will employ a controlled real-life experiment conducted in a cloud-based Kubernetes environment. The experiment is reproducible and quantitative. No human subjects are involved; only system performance metrics are collected.

Primary method: Laboratory experiment with repeated measures (each configuration is tested multiple times) and blocking on service type to account for heterogeneity.

2.8 Research tools

Experimental setup:

- **Cloud provider:** Local Kubernetes cluster
- **Orchestration:** Kubernetes (version 1.28+)
- **Service meshes:**
  - Istio (latest stable, e.g., 1.22) with default sidecar injection
  - Linkerd (latest stable, e.g., 2.15) with default sidecar injection
- **Custom microservice application:** Built by the team, consisting of:
  - **AI Service**: service with AI agent logic used for test data generation
  - **Media Service**: service for video splitting and digital filters
  - **Math Services**: C# implementations of differential equations, integration, permutations, Fibonacci (with configurable n)
- **Load generator:** A separate virtual machine (VM) running a custom Python script that sends HTTP requests to the ingress of the Kubernetes cluster. The script controls the request rate (25, 50, 100 req/s) and targets each service endpoint. The VM is isolated from the cluster to avoid affecting resource measurements.
- **Metrics collection:**
  - Prometheus (for system metrics)
  - Mesh-specific telemetry (Istio's built-in, Linkerd's viz extension)
  - Kubernetes metrics-server (for container CPU/memory)
  - Custom metrics for CPU clock scaling

Experiment design (detailed):

RQ1) Deploy baseline (no service mesh) - measure performance for each service type under three load levels (10 repetitions each).
RQ2) Deploy Istio with mTLS disabled - repeat measurements.
RQ3) Deploy Istio with mTLS enabled - repeat measurements.
RQ4) Deploy Linkerd with mTLS disabled - repeat measurements.
RQ5) Deploy Linkerd with mTLS enabled - repeat measurements.
RQ6) Each test run consists of:
- 60s warm-up
- 120s steady-state measurement
- Cooldown

RQ7) Record aggregated metrics per service type and configuration.

2.9 Expected results

Quantitative:

- Latency distributions (p50, p95, p99) per service type for each configuration and load level.
- Throughput saturation curves.
- CPU and memory overhead percentages relative to baseline, broken down by service type.
- CPU clock scaling behavior on math services (Fibonacci) under mesh vs. baseline.

Qualitative:

- Identification of bottlenecks specific to each mesh.
- Insights into whether overhead is constant or scales with load and service type.
- Recommendations for practitioners: which mesh to choose based on workload characteristics (CPU-intensive math, I/O media, or mixed).

2.10 Validity threats

Threat type	Description	Mitigation
Construct validity	Measured metrics may not fully represent "efficiency" (e.g., latency alone ignores user experience).	Use multiple metrics (latency, throughput, resource usage, clock scaling). Relate to SLR definitions.
Internal validity	Uncontrolled variables (e.g., network jitter, node scheduling) affect results.	Run all tests on isolated, dedicated cluster; repeat each condition 10 times; randomize order.
External validity	Results may not generalize to other applications, cloud providers, or service meshes.	Use a custom application that covers diverse workload types; test two representative meshes (Istio, Linkerd); discuss limitations.
Conclusion validity	Random chance may produce false significance.	Use appropriate statistical tests, set $\alpha = 0.05$, and report effect sizes and confidence intervals.

2.11 Research plan

Phase	Tasks	Responsible	Duration
RQ1) Application development	Build and containerize AI, Media, Math services; create Kubernetes deployment manifests	All team members	5 days
RQ2) Environment setup	Provision Kubernetes cluster, install Istio and Linkerd, deploy custom app, validate baseline	Oleksandr Nychyporchuk, Kiryl Pashkevich	3 days
RQ3) Load generator & monitoring	Develop custom load script on separate VM, configure Prometheus, create measurement scripts targeting each service	Pavel Khmialeuski, Alla Krylova	2 days

RQ4) Pilot study	Run 1 repetition of each configuration (13 runs) for one service type (Math) to validate toolchain	Alla Krylova (execution), Oleksandr Nychyporchuk (verification)	1 day
RQ5) Full experiment	Execute 10 repetitions per configuration × 3 service types (390 runs), collect raw data	All team members (divided runs)	7-10 days
RQ6) Data analysis	Statistical tests, visualization, hypothesis testing	Oleksandr Nychyporchuk, Kiryl Pashkevich	4 days
RQ7) Reporting	Write research paper / report sections	Alla Krylova (draft), Oleksandr Nychyporchuk (final)	4 days

2.12 Publication goals

The results of this research are suitable for publication in:

- **Conference:** IEEE International Conference on Cloud Computing (CLOUD), IEEE International Conference on Edge Computing (EDGE), or International Conference on Software Engineering (ICSE) - SEIP track.
- **Journal:** Journal of Systems and Software, IEEE Transactions on Network and Service Management, or Cluster Computing.

The pilot study results alone may serve as a short paper or a technical report. The full experiment, if significant, can be submitted as a full conference paper.

3 Pilot study

3.1 Research subjects

For the pilot study, we used a subset of the experimental configurations to verify that all tools work and produce plausible data. The pilot focused on the Mathematical Modules (specifically the Permutations service $O(n!)$) as the most CPU-intensive. The pilot included 7 configurations (single repetition each):

RQ1) Baseline - low, medium, high load (25, 50, 100 req/s) – 3 runs
RQ2) Istio (mTLS off) - low, medium, high load – 3 runs
RQ3) Linkerd (mTLS off) - high load only (100 req/s) – 1 run

We omitted mTLS-on conditions and other service types in the pilot due to time constraints but will include them in the full experiment.

Qualification: The pilot used the same hardware and software versions as planned for the full experiment.

3.2 Study execution

- **Executed by:**
- **Verified by:**
- **Date:**
- **Environment:**

- **Tools:**
- **Procedure:**

RQ1) Deploy baseline (no service mesh) application.
RQ2) From the load VM, run the script for each load level (25, 50, 100 req/s) - each test includes 30s warm-up, 60s measurement.
RQ3) Record latency (p95), CPU usage of the permutations container, and total memory.
RQ4) Install Istio with default sidecar injection, disable mTLS, redeploy application.
RQ5) Repeat step 2-3.
RQ6) Install Linkerd with default sidecar injection, disable mTLS, redeploy application.
RQ7) Repeat high-load test only (100 req/s).
RQ8) Aggregate metrics.

3.3 Results

3.4 Conclusions

4 Conclusions

4.1 Design

4.2 Pilot study

5 Literature

Bibliography

[1] A. B. Barr, O. Lavi, Y. Naor, S. Rampal, and J. Tavori, "Technical Report: Performance Comparison of Service Mesh Frameworks: the mTLS Test Case," technical report arXiv:2411.02267, 2024.

[2] M. Ganguli, S. Ranganath, S. Ravisundar, A. Layek, D. Ilangovan, and E. Verplanke, "Challenges and Opportunities in Performance Benchmarking of Service Mesh for the Edge," in *2021 IEEE International Conference on Edge Computing (EDGE)*, Chicago, IL, USA: IEEE, 2021, pp. 78–85. doi: 10.1109/EDGE53862.2021.00020.

[3] K. Bosquez, X. Caiza, L. Nunez, and J. Monar, "Comparative Evaluation of Linkerd and Istio Service Meshes in a Microservices Architecture Application," in *2025 IEEE Colombian Caribbean Conference (C3)*, Santa Marta, Colombia: IEEE, 2025, pp. 1–6. doi: 10.1109/C366505.2025.11340184.

[4] W. Yang, P. Chen, G. Yu, H. Zhang, and H. Zhang, "Network shortcut in data plane of service mesh with eBPF," *Journal of Network and Computer Applications*, vol. 222, p. 103805, 2024, doi: 10.1016/j.jnca.2023.103805.

[5] M. Nagy, T. Lévai, F. Németh, A. Panda, G. Antichi, and G. Rétvári, "Elastic Scaling of Real-Time Communication Services," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 3393–3405, 2026, doi: 10.1109/TNSM.2026.3674598.

[6] I. Dermentzis, G. Koukis, and V. Tsaoussidis, "Trade-Offs in Kubernetes Security and Energy Consumption," *Urban Science*, vol. 10, no. 2, p. 81, 2026, doi: 10.3390/urbansci10020081.

[7] H.-Y. Huang, Y.-Y. Fanjiang, C.-H. Hung, H.-Y. Tsai, and B.-H. Lin, "Evaluation of a Smart Intercom Microservice System Based on the Cloud of Things," *Electronics*, vol. 12, no. 11, p. 2406, 2023, doi: 10.3390/electronics12112406.

[8] S. Singh, C. H. Muntean, and S. Gupta, “Resilient microservices: an investigation into Istio effectiveness in Kubernetes,” *Cluster Computing*, vol. 29, no. 1, p. 27, 2026, doi: 10.1007/s10586-025-05750-x.

[9] L. Larsson, W. Tärneberg, C. Klein, E. Elmroth, and M. Kihl, “Impact of etcd Deployment on Kubernetes, Istio, and Application Performance,” technical report arXiv:2004.00372, 2020.

[10] D. A. Hahn, D. Davidson, and A. G. Bardas, “Security Issues and Challenges in Service Meshes – An Extended Study,” technical report arXiv:2010.11079, 2020.

[11] Q. Zeng, M. Kavousi, Y. Luo, L. Jin, and Y. Chen, “Full-stack vulnerability analysis of the cloud-native platform,” *Computers & Security*, vol. 129, p. 103173, 2023, doi: 10.1016/j.cose.2023.103173.

[12] S. Kurpad, S. Bt, S. Vijaykumar, S. Jain, and S. Kalambur, “Microarchitectural Analysis and Characterization of Performance Overheads in Service Meshes with Kubernetes,” in *2023 3rd Asian Conference on Innovation in Technology (ASIANCON)*, Ravet IN, India: IEEE, 2023, pp. 1–6. doi: 10.1109/ASIANCON58793.2023.10270428.